Krylov Sobolev PDE Benchmark

Natural language statement	Domain	GroundTruth

Lemma 1.1.1. Let u in C02 be a solution of lambdau - Deltau = f in R^d. Then lambda2||u||2_{L2} + 2lambda sum_{j=1}^d ||u_{x^j}||2_{L2} + sum_{j,k=1}^d ||u_{x^jx^k}||2_{L2} = ||f||2_{L2}. (Here lambda > 0 is fixed, C02 is the set of twice continuously differentiable functions on R^d with compact support, and Deltau = u_{x1x1} + ... + u_{x^dx^d}.)

Domain: PDE

```
noncomputable def directionalDerivativeList {d : N} :
    List (Fin d) -> (EuclideanSpace R (Fin d) -> R) -> EuclideanSpace R
    (Fin d) -> R
  | [], u => u
  | i :: indices, u => fun x =>
      fderiv R (directionalDerivativeList indices u) x
      (EuclideanSpace.single i 1)

theorem krylov_sobolev_1_1_1_energy_identity {d : N} (lam : R) (hlam : 0 <
lam)
    (u f : EuclideanSpace R (Fin d) -> R)
    (hu : ContDiff R 2 u) (hsupp : HasCompactSupport u)
    (heq : forall x, lam * u x - Delta u x = f x) :
    lam ^ 2 * (integral x, u x ^ 2) +
          2 * lam * (sum j, integral x, directionalDerivativeList [j] u x ^
        2) +
          (sum j, sum k, integral x, directionalDerivativeList [j, k] u x ^
        2) =
        integral x, f x ^ 2 := by
  sorry
```

Exercise 1.1.3. By using the Fourier transform, prove that, if d = 2 and u in C02, then integral_{R^d} det u_{xx} dx = 0. (Here u_{xx} = (u_{x^ix^j}) is the Hessian matrix of u and C02 is the set of twice continuously differentiable functions with compact support.)

Domain: PDE

```
noncomputable def hessian {d : N} (u : EuclideanSpace R (Fin d) -> R)
    (x : EuclideanSpace R (Fin d)) : Matrix (Fin d) (Fin d) R :=
  Matrix.of fun i j => directionalDerivativeList [i, j] u x

theorem krylov_sobolev_1_1_3_hessian_determinant_integral
    (u : EuclideanSpace R (Fin 2) -> R) (hu : ContDiff R 2 u) (hsupp :
    HasCompactSupport u) :
    integral x, (hessian u x).det = 0 := by
  sorry
```

Lemma 1.1.7. Let lambda > 0 and let u be a bounded from above twice continuously differentiable function on R^d satisfying Deltau - lambdau >= 0 in R^d. Then u <= 0. In particular, if u is bounded and Deltau - lambdau = 0, then u <= 0, so that u == 0.

Domain: PDE

```
theorem krylov_sobolev_1_1_7_whole_space_maximum_principle {d : N} (lam :
R) (hlam : 0 < lam) :
    (forall u : EuclideanSpace R (Fin d) -> R, ContDiff R 2 u -> BddAbove
    (range u) ->
        (forall x, 0 <= Delta u x - lam * u x) -> forall x, u x <= 0) /\
      forall u : EuclideanSpace R (Fin d) -> R, ContDiff R 2 u ->
      Bornology.IsBounded (range u) ->
        (forall x, Delta u x - lam * u x = 0) -> u = 0 := by
  sorry
```

Theorem 1.5.1. Let Omega = R^d or Omega = R^d_+. For any p in [1,inf) and u in W_p2(Omega) we have ||u_x||_{L_p(Omega)} <= N ||u||^{1/2}_{L_p(Omega)} ||u_{xx}||^{1/2}_{L_p(Omega)}, where N is independent of u. (Here u_x = grad u, u_{xx} = (u_{x^ix^j}), the derivatives are generalized (Sobolev) derivatives, and u in W_p^k(Omega) means that u and all D^alpha u with |alpha| <= k belong to L_p(Omega).)

Domain: PDE

```
def upperHalfSpace (d : N) [NeZero d] : Opens (EuclideanSpace R (Fin d))
where
  carrier := {x | 0 < x 0}
  is_open' := isOpen_lt continuous_const (by fun_prop)

def IsSobolevFamilyOn {d : N} (k : N) (Omega : Opens (EuclideanSpace R (Fin
d)))
    (u : EuclideanSpace R (Fin d) -> R) (D : (Fin d -> N) -> EuclideanSpace
    R (Fin d) -> R) : Prop :=
  forall alpha in multiIndicesLE d k, IsSobolevDerivOn Omega alpha u (D
  alpha)

def MemSobolevOn {d : N} (p : R>=0inf) (k : N) (Omega : Opens
(EuclideanSpace R (Fin d)))
    (u : EuclideanSpace R (Fin d) -> R) : Prop :=
  MemLp u p (volume.restrict Omega) /\
    exists D, IsSobolevFamilyOn k Omega u D /\ forall alpha in
    multiIndicesLE d k,
      MemLp (D alpha) p (volume.restrict Omega)

noncomputable def gradNorm {d : N} (D : (Fin d -> N) -> EuclideanSpace R
(Fin d) -> R)
    (x : EuclideanSpace R (Fin d)) : R :=
  (sum i, D (Pi.single i 1) x ^ 2)

noncomputable def hessNorm {d : N} (D : (Fin d -> N) -> EuclideanSpace R
(Fin d) -> R)
    (x : EuclideanSpace R (Fin d)) : R :=
  (sum i, sum j, D (Pi.single i 1 + Pi.single j 1) x ^ 2)
```

<table>
<tr><th>Natural language statement</th><th>Domain</th><th>GroundTruth</th></tr>
<tr><td></td><td></td><td>theorem krylov_sobolev_1_5_1_multiplicative_inequality {d : N} [NeZero d]
(p : R)
    (hp : 1 <= p) (Omega : TopologicalSpace.Opens (EuclideanSpace R (Fin
    d)))
    (hOmega : Omega = top \/ Omega = upperHalfSpace d) :
    exists N : R>=0inf, N != (inf : R>=0inf) /\
      forall (u : EuclideanSpace R (Fin d) -> R) (D : (Fin d -> N) ->
      EuclideanSpace R (Fin d) -> R),
        MemSobolevOn (ENNReal.ofReal p) 2 Omega u -> IsSobolevFamilyOn 2
        Omega u D ->
        eLpNorm (gradNorm D) (ENNReal.ofReal p) (volume.restrict Omega) <=
          N * eLpNorm u (ENNReal.ofReal p) (volume.restrict Omega) ^ (2^-1
        : R) *
            eLpNorm (hessNorm D) (ENNReal.ofReal p) (volume.restrict Omega)
        ^ (2^-1 : R) := by
  sorry</td></tr>
<tr><td>Theorem 3.2.10 (Fefferman-Stein). Let p in (1,inf). Then for any f in L_p(Omega) we have ||f||_{L_p(Omega)} <= N ||f^#||_{L_p(Omega)}, where N = (2q)^p N0^{p-1}, q = p/(p-1). (Here (Omega,F,mu) is a complete measure space with a sigma-finite measure mu such that mu(Omega) = inf; (C_n, n in Z) is a filtration of partitions of Omega with regularity constant N0, f_{|n}(x) = _{C_n(x)} f dmu is the average of f over the element C_n(x) of the n-th partition containing x, and f^#(x) = sup_{n<inf} _{C_n(x)} |f(y) - f_{|n}(y)| mu(dy).)</td><td>Real analysis</td><td>structure FiltrationOfPartitions (X : Type*) [MeasurableSpace X] (mu :
Measure X) where
  /-- ‘cell n x‘ is the element ‘C_n(x)‘ of the ‘n‘-th partition containing
  ‘x‘. -/
  cell : Z -> X -> Set X
  countable_partition : forall n, (range (cell n)).Countable
  measurableSet_cell : forall n x, MeasurableSet (cell n x)
  measure_cell_ne_top : forall n x, mu (cell n x) != (inf : R>=0inf)
  mem_cell : forall n x, x in cell n x
  cell_congr : forall n x y, y in cell n x -> cell n y = cell n x
  cell_subset : forall n x, cell n x subset cell (n - 1) x
  /-- The regularity constant ‘N0‘ of condition (iii). -/
  regularityConstant : R>=0inf
  measure_cell_le : forall n x, mu (cell (n - 1) x) <= regularityConstant *
  mu (cell n x)
  tendsto_measure_atBot : Tendsto (fun n : Z => x, mu (cell n x)) atBot
  (nhds (inf : R>=0inf))
  /-- The dense subset ‘L subset L1(Omega)‘ of condition (i). -/
  denseClass : Set (X -> R)
  denseClass_dense : forall f : X -> R, MemLp f 1 mu -> forall eps :
  R>=0inf, 0 < eps ->
    exists g in denseClass, MemLp g 1 mu /\ eLpNorm (f - g) 1 mu < eps
  denseClass_differentiation : forall g in denseClass, forall^m x dmu,
    Tendsto (fun n : Z => y in cell n x, g y dmu) atTop (nhds (g x))

noncomputable def sharpFunction {X : Type*} [MeasurableSpace X] {mu :
Measure X}
    (F : FiltrationOfPartitions X mu) (f : X -> R) (x : X) : R>=0inf :=
   n : Z, ENNReal.ofReal (y in F.cell n x, |f y - z in F.cell n y, f z
   dmu| dmu)

theorem krylov_sobolev_3_2_10_fefferman_stein {X : Type*} [MeasurableSpace
X]
    (mu : Measure X) [SigmaFinite mu] [mu.IsComplete] (hmu : mu univ = (inf
    : R>=0inf))
    (F : FiltrationOfPartitions X mu) (p : R) (hp : 1 < p)
    (f : X -> R) (hf : MemLp f (ENNReal.ofReal p) mu) :
    eLpNorm f (ENNReal.ofReal p) mu <=
      ENNReal.ofReal ((2 * (p / (p - 1))) ^ p) * F.regularityConstant ^ (p
      - 1) *
        eLpNorm’ (sharpFunction F f) p mu := by
  sorry</td></tr>
<tr><td>Theorem 10.2.1 (Morrey). Let Omega be a bounded convex domain in R^d satisfying diam Omega <= kappa rho(Omega) with a constant kappa < inf, where rho(Omega) is the largest diameter of open balls contained in Omega. Let M < inf, alpha in (0,1] be some constants. Finally, let u, u_x in L1(Omega) and assume that for any ball B subset Omega we have _B |u_x| dx <= M r_B^{alpha-1}, where r_B is the radius of B. Then for any x, y in Omega we have |u(x) - u(y)| <= NM|x-y|^alpha and |u(x)| <= NM + sup_{yinOmega} _{OmegainterB1(y)} |u(z)| dz, where N depends only on d, alpha, and kappa. In particular, if p > d and u, u_x in L_p(Omega), then both estimates hold with alpha</td><td>PDE</td><td>def IsSobolevFamilyOn {d : N} (k : N) (Omega : Opens (EuclideanSpace R (Fin
d)))
    (u : EuclideanSpace R (Fin d) -> R) (D : (Fin d -> N) -> EuclideanSpace
    R (Fin d) -> R) : Prop :=
  forall alpha in multiIndicesLE d k, IsSobolevDerivOn Omega alpha u (D
  alpha)

def MemSobolevOn {d : N} (p : R>=0inf) (k : N) (Omega : Opens
(EuclideanSpace R (Fin d)))
    (u : EuclideanSpace R (Fin d) -> R) : Prop :=
  MemLp u p (volume.restrict Omega) /\</td></tr>
</table>

Natural language statement	Domain	GroundTruth

= 1 - d/p, M = ||u_x||_{L_p(Omega)} and N depending only on d, p, and kappa. (As the proof states, the assertion is that there is a modification of u, i.e. a function equal to u almost everywhere, satisfying the two estimates.)		`exists D, IsSobolevFamilyOn k Omega u D /\ forall alpha in multiIndicesLE d k,`
		`MemLp (D alpha) p (volume.restrict Omega)`
		
		`noncomputable def gradNorm {d : N} (D : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R)`
		`(x : EuclideanSpace R (Fin d)) : R :=`
		`(sum i, D (Pi.single i 1) x ^ 2)`
		
		`structure IsConvexDomainWith {d : N} (kappa : R) (Omega : Opens (EuclideanSpace R (Fin d))) : Prop where`
		`nonempty : (Omega : Set (EuclideanSpace R (Fin d))).Nonempty`
		`convex : Convex R (Omega : Set (EuclideanSpace R (Fin d)))`
		`isBounded : Bornology.IsBounded (Omega : Set (EuclideanSpace R (Fin d)))`
		`diam_le : Metric.diam (Omega : Set (EuclideanSpace R (Fin d))) <= kappa * interiorDiameter Omega`
		
		`noncomputable def unitBallAverageSup {d : N} (Omega : Set (EuclideanSpace R (Fin d)))`
		`(u : EuclideanSpace R (Fin d) -> R) : R>=0inf :=`
		`y : Omega, ENNReal.ofReal`
		`( z in Omega inter Metric.ball (y : EuclideanSpace R (Fin d)) 1, |u z|)`
		
		`theorem krylov_sobolev_10_2_1_morrey_embedding {d : N} (kappa : R) :`
		`(forall alpha : R, 0 < alpha -> alpha <= 1 -> exists N : R, 0 <= N /\`
		`forall (Omega : TopologicalSpace.Opens (EuclideanSpace R (Fin d)))`
		`(u : EuclideanSpace R (Fin d) -> R)`
		`(D : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R) (M : R),`
		`IsConvexDomainWith kappa Omega -> 0 <= M -> MemSobolevOn 1 1 Omega u ->`
		`IsSobolevFamilyOn 1 Omega u D ->`
		`(forall (z : EuclideanSpace R (Fin d)) (r : R), 0 < r ->`
		`Metric.ball z r subset (Omega : Set (EuclideanSpace R (Fin d))) ->`
		`x in Metric.ball z r, gradNorm D x <= M * r ^ (alpha - 1)) ->`
		`exists v : EuclideanSpace R (Fin d) -> R, v =^m[volume.restrict Omega] u /\`
		`(forall x in Omega, forall y in Omega, |v x - v y| <= N * M * ||x - y|| ^ alpha) /\`
		`forall x in Omega, ENNReal.ofReal |v x| <=`
		`ENNReal.ofReal (N * M) + unitBallAverageSup (Omega : Set (EuclideanSpace R (Fin d))) u) /\`
		`forall p : R, 1 <= p -> (d : R) < p -> exists N : R, 0 <= N /\`
		`forall (Omega : TopologicalSpace.Opens (EuclideanSpace R (Fin d)))`
		`(u : EuclideanSpace R (Fin d) -> R)`
		`(D : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R),`
		`IsConvexDomainWith kappa Omega -> MemSobolevOn (ENNReal.ofReal p) 1 Omega u ->`
		`IsSobolevFamilyOn 1 Omega u D ->`
		`exists v : EuclideanSpace R (Fin d) -> R, v =^m[volume.restrict Omega] u /\`
		`(forall x in Omega, forall y in Omega, |v x - v y| <=`
		`N * (eLpNorm (gradNorm D) (ENNReal.ofReal p) (volume.restrict Omega)).toReal *`
		`||x - y|| ^ (1 - d / p)) /\`
		`forall x in Omega, ENNReal.ofReal |v x| <=`
		`ENNReal.ofReal (N * (eLpNorm (gradNorm D) (ENNReal.ofReal p)`
		`(volume.restrict Omega)).toReal) +`
		`unitBallAverageSup (Omega : Set (EuclideanSpace R (Fin d)))`
		`u := by`
		`sorry`

Theorem 10.3.2 (Gagliardo-Nirenberg). Let Omega = R^d or Omega = R^d_+. Let u in W11(Omega). Then u in L_{d/(d-1)}(Omega) and ||u||_{L_{d/(d-1)}(Omega)} <= prod_{j=1}^d ||D_j u||_{L1(Omega)}^{1/d}. (The exponent d/(d-1) is read as inf when d = 1, the case with which the proof's induction starts.)	PDE	`def upperHalfSpace (d : N) [NeZero d] : Opens (EuclideanSpace R (Fin d))`
		`where`
		`carrier := {x | 0 < x 0}`
		`is_open' := isOpen_lt continuous_const (by fun_prop)`
		
		`def IsSobolevFamilyOn {d : N} (k : N) (Omega : Opens (EuclideanSpace R (Fin d)))`
		`(u : EuclideanSpace R (Fin d) -> R) (D : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R) : Prop :=`

Natural language statement	Domain	GroundTruth

		`forall alpha in multiIndicesLE d k, IsSobolevDerivOn Omega alpha u (D alpha)`
		
		`def MemSobolevOn {d : N} (p : R>=0inf) (k : N) (Omega : Opens (EuclideanSpace R (Fin d)))`
		`(u : EuclideanSpace R (Fin d) -> R) : Prop :=`
		`MemLp u p (volume.restrict Omega) /\`
		`exists D, IsSobolevFamilyOn k Omega u D /\ forall alpha in multiIndicesLE d k,`
		`MemLp (D alpha) p (volume.restrict Omega)`
		
		`theorem krylov_sobolev_10_3_2_gagliardo_nirenberg {d : N} [NeZero d]`
		`(Omega : TopologicalSpace.Opens (EuclideanSpace R (Fin d)))`
		`(hOmega : Omega = top \/ Omega = upperHalfSpace d) (u : EuclideanSpace R (Fin d) -> R)`
		`(D : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R) (hu : MemSobolevOn 1 1 Omega u)`
		`(hD : IsSobolevFamilyOn 1 Omega u D) :`
		`MemLp u ((d : R>=0inf) / ((d : R>=0inf) - 1)) (volume.restrict Omega) /\`
		`eLpNorm u ((d : R>=0inf) / ((d : R>=0inf) - 1)) (volume.restrict Omega) <=`
		`prod j, eLpNorm (D (Pi.single j 1)) 1 (volume.restrict Omega) ^ ((d : R)^-1) := by`
		`sorry`

Theorem 10.5.1 (Kondrashov). Let p in [1,inf), k in {1,2,...}, m in {0,1,...,k-1}, Omega in C^k, and let U be a bounded subset of W_p^k(Omega). Then U is precompact in W_q^m(Omega) for any q in [1,inf) satisfying the strict inequality k - d/p > m - d/q, so that q is just any number in [1,inf) if p(k-m) >= d. (Omega in C^k is Definition 8.3.1: Omega is a bounded domain and there are K0, rho0 > 0 such that every z in dOmega admits a one-to-one map psi of B_{rho0}(z) onto a domain D^z with psi(B_{rho0}(z) inter Omega) subset R^d_+, psi(z) = 0, psi(B_{rho0}(z) inter dOmega) = D^z inter {y1 = 0}, psi in C^k(closure B_{rho0}(z)), psi^{-1} in C^k(closure D^z) and |psi|_{C^k} + |psi^{-1}|_{C^k} <= K0.)	PDE	`def multiIndicesLE (d k : N) : Finset (Fin d -> N) :=`
		`(Fintype.piFinset fun _ : Fin d => Finset.range (k + 1)).filter fun alpha => sum i, alpha i <= k`
		
		`def IsSobolevFamilyOn {d : N} (k : N) (Omega : Opens (EuclideanSpace R (Fin d)))`
		`(u : EuclideanSpace R (Fin d) -> R) (D : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R) : Prop :=`
		`forall alpha in multiIndicesLE d k, IsSobolevDerivOn Omega alpha u (D alpha)`
		
		`def MemSobolevOn {d : N} (p : R>=0inf) (k : N) (Omega : Opens (EuclideanSpace R (Fin d)))`
		`(u : EuclideanSpace R (Fin d) -> R) : Prop :=`
		`MemLp u p (volume.restrict Omega) /\`
		`exists D, IsSobolevFamilyOn k Omega u D /\ forall alpha in multiIndicesLE d k,`
		`MemLp (D alpha) p (volume.restrict Omega)`
		
		`noncomputable def sobolevNorm {d : N} (p : R>=0inf) (k : N)`
		`(Omega : Opens (EuclideanSpace R (Fin d)))`
		`(D : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R) : R>=0inf :=`
		`sum alpha in multiIndicesLE d k,`
		`eLpNorm (D alpha) p (volume.restrict Omega)`
		
		`structure IsCkDomain {d : N} (k : N) (Omega : Opens (EuclideanSpace R (Fin d))) : Prop where`
		`nonempty : (Omega : Set (EuclideanSpace R (Fin d))).Nonempty`
		`isBounded : Bornology.IsBounded (Omega : Set (EuclideanSpace R (Fin d)))`
		`/-- Near every boundary point some ‘C^k‘ diffeomorphism flattens the boundary, with uniform`
		`bounds ‘K0‘ on the ‘C^k‘ norms and a uniform radius ‘rho0‘. -/`
		`exists_flattening : exists K0 rho0 : R, 0 < K0 /\ 0 < rho0 /\`
		`forall z in frontier (Omega : Set (EuclideanSpace R (Fin d))),`
		`exists (D : Opens (EuclideanSpace R (Fin d)))`
		`(psi psiinv : EuclideanSpace R (Fin d) -> EuclideanSpace R (Fin d)),`
		`BijOn psi (Metric.ball z rho0) D /\ InvOn psiinv psi (Metric.ball z rho0) D /\ psi z = 0 /\`
		`psi ’’ (Metric.ball z rho0 inter (Omega : Set (EuclideanSpace R (Fin d)))) subset`
		`(upperHalfSpaceSet d) /\`
		`psi ’’ (Metric.ball z rho0 inter frontier (Omega : Set (EuclideanSpace R (Fin d)))) =`
		`(D : Set (EuclideanSpace R (Fin d))) inter boundaryHyperplane d /\`
		`ContDiffOn R k psi (Metric.closedBall z rho0) /\`

Natural language statement	Domain	GroundTruth

		`ContDiffOn R k psiinv (closure (D : Set (EuclideanSpace R (Fin d)))) /\ ckNormMapOn k (Metric.closedBall z rho0) psi + ckNormMapOn k (closure (D : Set (EuclideanSpace R (Fin d)))) psiinv <= ENNReal.ofReal K0`
		
		`theorem krylov_sobolev_10_5_1_kondrashov_compactness {d : N} (p q : R) (k m : N) (hp : 1 <= p) (hq : 1 <= q) (hm : m < k) (Omega : TopologicalSpace.Opens (EuclideanSpace R (Fin d))) (hOmega : IsCkDomain k Omega) (hkm : (m : R) - d / q < (k : R) - d / p) (U : Set (EuclideanSpace R (Fin d) -> R)) (hU : forall u in U, MemSobolevOn (ENNReal.ofReal p) k Omega u) (hbdd : exists M : R>=0inf, M != (inf : R>=0inf) /\ forall u in U, forall D, IsSobolevFamilyOn k Omega u D -> sobolevNorm (ENNReal.ofReal p) k Omega D <= M) (F : N -> EuclideanSpace R (Fin d) -> R) (hF : forall n, F n in U) (DF : N -> (Fin d -> N) -> EuclideanSpace R (Fin d) -> R) (hDF : forall n, IsSobolevFamilyOn m Omega (F n) (DF n)) : exists (sigma : N -> N) (g : EuclideanSpace R (Fin d) -> R) (Dg : (Fin d -> N) -> EuclideanSpace R (Fin d) -> R), StrictMono sigma /\ MemSobolevOn (ENNReal.ofReal q) m Omega g /\ IsSobolevFamilyOn m Omega g Dg /\ Tendsto (fun n => sum alpha in multiIndicesLE d m, eLpNorm (DF (sigma n) alpha - Dg alpha) (ENNReal.ofReal q) (volume.restrict Omega)) atTop (nhds 0) := by sorry`

Theorem 11.1.3 (Maximum principle). Let Omega be a bounded domain, c <= 0. If u in C2_{loc}(Omega) inter C(closure Omega) and Lu >= 0 in Omega, then u <= max_{dOmega} u+ holds in Omega. (Here L = a^{ij}D_{ij} + b^iD_i + c is a second-order elliptic differential operator in the sense of Definition 1.4.1: a^{ij}, b^i, c are real-valued measurable functions with a^{ij} = a^{ji} and there is a constant of ellipticity kappa > 0 with kappa^{-1}|xi|2 >= a^{ij}(x)xi^ixi^j >= kappa|xi|2 for all x, xi; by Assumption 1.6.1 the coefficients are bounded. Section 11.1 notes that the uniform continuity of a^{ij} is not used. Also a+ = (|a| + a)/2.)	PDE	`structure EllipticCoefficients (d : N) (kappa K : R) where /-- The leading coefficients ‘a^{ij}‘. -/ a : EuclideanSpace R (Fin d) -> Matrix (Fin d) (Fin d) R /-- The first-order coefficients ‘b^i‘. -/ b : EuclideanSpace R (Fin d) -> EuclideanSpace R (Fin d) /-- The zeroth-order coefficient ‘c‘. -/ c : EuclideanSpace R (Fin d) -> R measurable_a : forall i j, Measurable fun x => a x i j measurable_b : forall i, Measurable fun x => b x i measurable_c : Measurable c isSymm : forall x, (a x).IsSymm ellipticityConstant_pos : 0 < kappa le_quadraticForm : forall (x xi : EuclideanSpace R (Fin d)), kappa * ||xi|| ^ 2 <= sum i, sum j, a x i j * xi i * xi j quadraticForm_le : forall (x xi : EuclideanSpace R (Fin d)), sum i, sum j, a x i j * xi i * xi j <= kappa^-1 * ||xi|| ^ 2 abs_a_le : forall x i j, |a x i j| <= K abs_b_le : forall x i, |b x i| <= K abs_c_le : forall x, |c x| <= K`  `noncomputable def EllipticCoefficients.op {d : N} {kappa K : R} (L : EllipticCoefficients d kappa K) (u : EuclideanSpace R (Fin d) -> R) (x : EuclideanSpace R (Fin d)) : R := (sum i, sum j, L.a x i j * directionalDerivativeList [i, j] u x) + (sum i, L.b x i * directionalDerivativeList [i] u x) + L.c x * u x`  `theorem krylov_sobolev_11_1_3_maximum_principle {d : N} (hd : 1 <= d) {kappa K : R} (L : EllipticCoefficients d kappa K) (hc : forall x, L.c x <= 0) (Omega : Set (EuclideanSpace R (Fin d))) (hOmegaopen : IsOpen Omega) (hOmegabdd : Bornology.IsBounded Omega) (u : EuclideanSpace R (Fin d) -> R) (hu : ContDiffOn R 2 u Omega) (hucont : ContinuousOn u (closure Omega)) (hLu : forall x in Omega, 0 <= L.op u x) : forall x in Omega, u x <= sSup ((fun y => max (u y) 0) ’’ frontier Omega) := by sorry`
Theorem 12.9.12. If > 0, then for any phi in S we have (1-Delta)^{-/2}phi(x) = integral_{R^d} G(x-y)phi(y) dy and G	Harmonic analysis	`def IsBesselPotential {d : N} ( : R) (phi v : EuclideanSpace R (Fin d) -> C) : Prop :=`

<table>
<tr><th>Natural language statement</th><th>Domain</th><th>GroundTruth</th></tr>
<tr><td>is a function depending only on |x|. Furthermore, G >= 0 and ||G||_{L1} = 1. (Here S is the Schwartz space and, by Definition 12.9.1, (1-Delta)^{/2} is the pseudo-differential operator of order with symbol (1+|xi|2)^{/2}, the Fourier transform being normalized as F(u)(xi) = (2pi)^{-d/2} integral e^{-ixi.x} u(x) dx.)</td><td></td><td> forall xi : EuclideanSpace R (Fin d),
    F v xi = ((1 + 4 * Real.pi ^ 2 * ||xi|| ^ 2) ^ (- / 2) : R) * F phi xi

theorem krylov_sobolev_12_9_12_bessel_kernel {d : N} (: R) (h : 0 <) :
    exists G : EuclideanSpace R (Fin d) -> R,
      (forall x y, ||x|| = ||y|| -> G x = G y) /\ (forall x, 0 <= G x) /\
        eLpNorm G 1 volume = 1 /\
        forall (phi : S(EuclideanSpace R (Fin d), C)) (v : EuclideanSpace R
        (Fin d) -> C),
          Continuous v -> Integrable v -> IsBesselPotential (fun x => phi
        x) v ->
            forall x, v x = integral y, (G (x - y) : C) * phi y := by
  sorry</td></tr>
</table>